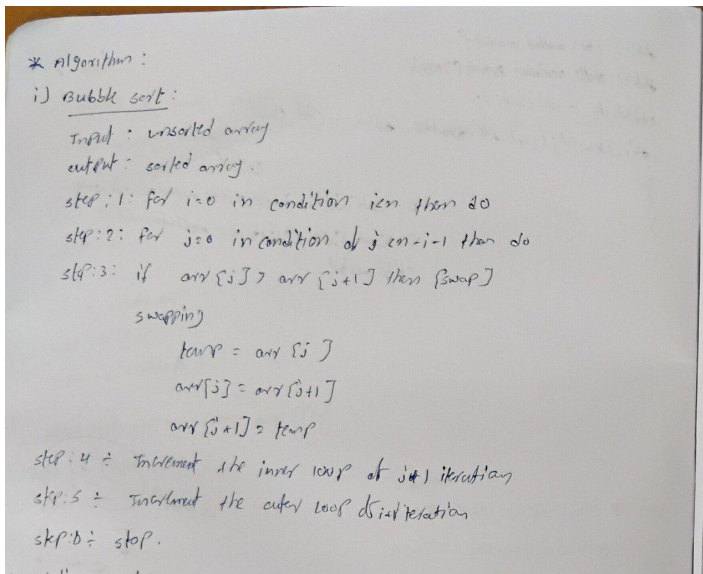


PRACTICAL-01

AIM: To implementation and time analysis of sorting algorithms. Bubble sort, Selection Sort, Insertion sort, Merge sort and Quick sort.

1.BUBBLE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

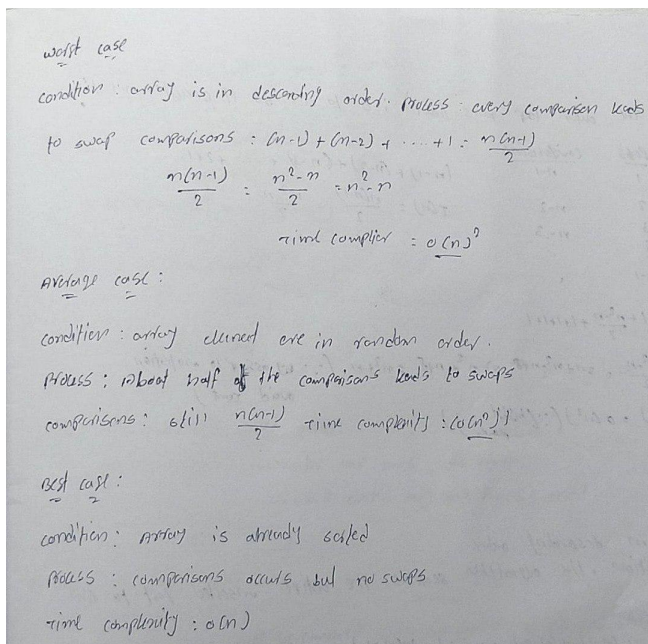
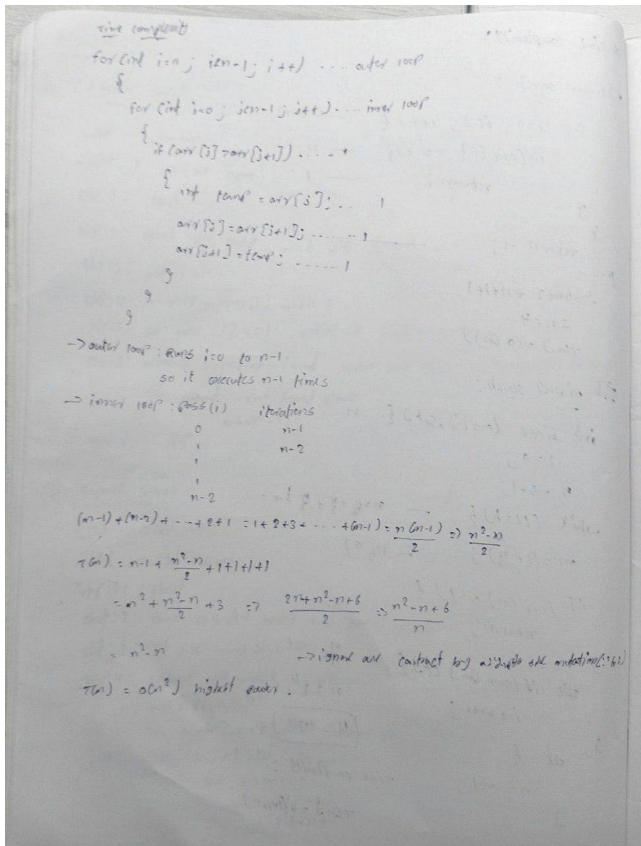
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
}  
  
int main() {  
    int n;  
  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
    cout << "Enter array elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    bubbleSort(arr, n);  
  
    cout << "Sorted elements: ";  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
  
    return 0;  
}
```

OUTPUT:

```
Enter number of elements: 5  
Enter array elements: 7  
2  
9  
1  
5  
Sorted elements: 1 2 5 7 9
```

TIME COMPLEXITY:



2. SELECTION SORT

ALGORITHM:

selection sort:
input: unsorted array
output: sorted array.
step 1: for i=0 to n condition i<n then do
step 2: min=i and inner for loop initialize j=i+1 to n
step 3: in the condition j<n then do if arr[j] < arr[min] then
min=j
step 4: increment the inner loop j++
step 5: if min!=i then
swap(arr[i], arr[min])
step 6: increment the outer loop i++
step 7: stop.

PROGRAM:

```
#include <iostream>
```

```
using namespace std;
```

```
void selectionSort(int arr[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        int minIndex = i;
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (arr[j] < arr[minIndex]) {
```

```
                minIndex = j;
```

```
            }
```

```
        }
```

```
        int temp = arr[minIndex];
```

```
        arr[minIndex] = arr[i];
```

```
        arr[i] = temp;
```

```
    }
```

```
}
```

```
int main() {
```

```
m.hima sankar(92460118218)
```

```
int n;

cout << "Enter number of elements: ";

cin >> n;

int arr[n];

cout << "Enter " << n << " elements: ";

for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Original array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
    cout << endl;
}

selectionSort(arr, n);

cout << "Sorted array: ";

for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
    //cout << endl;
}

return 0;
}
```


OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 9
2
4
6
0
Original array: 9
2
4
6
0
Sorted array: 0 2 4 6 9
```

TIME COMPLEXITY:

solution sort

```
for (int i=0, i<n-1; i++)
{
    int min index = i;
    for (int j=i+1; j<n; j++)
    {
        if (arr[j] < arr[min index])
        {
            min index = j;
        }
    }
    int temp = arr[min index];
    arr[min index] = arr[i];
    arr[i] = temp;
}
```

outer loop: The outer loop runs from $i=0$ to $n-1$ so it executes $n-1$ times.

inner loop: steps comparisons

steps	comparisons
1	$n-1$
2	$n-2$
3	$n-3$
...	...
$n-1$	1

$(n-1) + (n-2) + (n-3) + \dots + 1$

$T(n) = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$

$T(n) = n-1 + \frac{n^2-n}{2} + 1 + 1 + 1$

$= n + \frac{n^2-n}{2} = \frac{2n + n^2 - n}{2} = \frac{n^2 + n}{2}$

$T(n) = O(n^2)$ (because of n^2 term)

worst case

→ array is in descending order.

→ for each position, the algorithm searches the entire unsorted part to find the minimum.

→ comparisons: $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$

→ swaps: At most $n-1$

→ time complexity: $O(n^2)$

average case

→ array elements are in random order.

→ still scans the unsorted part fully for each position.

→ comparisons: same as worst case $\frac{n(n-1)}{2}$

→ swaps: $n-1$

→ time complexity: $O(n^2)$

best case

→ array is already sorted.

→ even though no swaps are needed, the algorithm still scans the unsorted part.

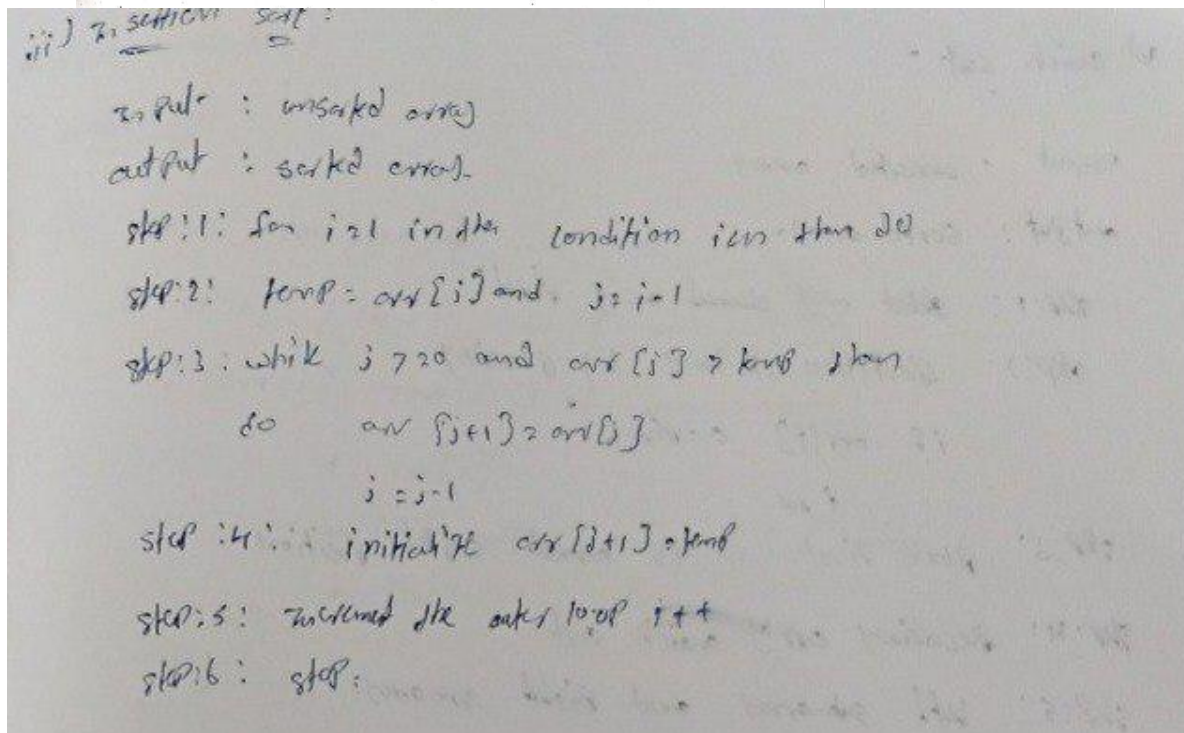
→ comparisons: same as worst case, $\frac{n(n-1)}{2}$

→ swaps: 0

→ time complexity: $O(n^2)$

3.INSERTION SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

```
}
```

```
int main() {  
    int n;  
    cout << "Enter number of elements: ";  
    cin >> n;  
  
    int arr[n];  
    cout << "Enter " << n << " elements: ";  
    for (int i = 0; i < n; i++) {  
        cin >> arr[i];  
    }  
  
    cout << "Original array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
        cout << endl;  
    }  
    insertionSort(arr, n);  
  
    cout << "Sorted array: ";  
    for (int i = 0; i < n; i++)  
    {  
        cout << arr[i] << " ";  
        //cout << endl;  
    }  
    return 0;  
}
```


OUTPUT:

```
Enter number of elements: 5
Enter 5 elements: 7
3
8
2
0
Original array: 7
3
8
2
0
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:

Insertion sort

```
for (int i = 1; i < n; i++)
{
    int key = arr[i];
    int j = i - 1;
    while (j > 0 && arr[j] > key)
    {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
```

1. Best case: already sorted array.
[1, 2, 3, 4, 5, 6] → only one pass, n-1 times
→ while checks only once for each element
 $T(n) = O(n)$

2. Worst case: reverse sorted array.
[5, 4, 3, 2, 1] → each new element must be compared with all previous elements and shifted.
Pass | Comparison
1 | 2
2 | 3
3 | 4
4 | 5
n-1 | n-1
 $1 + 2 + 3 + \dots + (n-1)$
 $T(n) = \frac{n(n-1)}{2} \approx \frac{n^2}{2} \Rightarrow O(n^2)$

Worst case

- array is in descending order.
- each new element must be compared with all previously sorted elements and shifted
- comparisons & shifts: $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$
- time complexity: $O(n^2)$

Average case

- array element are in random order
- on average each element is compared with half of the sorted portion.
- comparisons: about $\frac{n(n-1)}{4}$
- time complexity: $O(n^2)$

Best case

- array is already sorted
- each element is compared only once and no shifting needed
- comparisons: $n-1$
- time complexity: $O(n)$

4. QUICK SORT

✓ quick sort :

input : unsorted array.

output : sorted array.

step:1: select any element as a pivot.

step:2: Partition the array by placing

if $arr[j] < pivot$

$i++$

step:3: place pivot in its correct sorted position

step:4: recursively apply quick sort

step:5: left sub-array and right sub-array

step:6: stop.

PROGRAM:

```
#include <iostream>
using namespace std;

int partition(int arr[], int lb, int ub) { int
    pivot = arr[lb];
    int start = lb; int
    end = ub;

    while (start < end) {
        while (arr[start] <= pivot ) {
            start++;
        }
        while (arr[end] > pivot) {
            end--;
        }
    }
}
```

```
}

    if (start < end) {
        swap(arr[start], arr[end]);
    }
}

swap(arr[lb], arr[end]);
return end;
}

void quickSort(int arr[], int lb, int ub) { if
    (lb < ub) {
        int loc = partition(arr, lb, ub);
        quickSort(arr, lb, loc - 1); quickSort(arr,
            loc + 1, ub);
    }
}

int main() {
    int n;
    cout << "Enter size of array: "; cin
    >> n;

    int arr[n];
    cout << "Enter " << n << " elements: "; for
    (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}
```

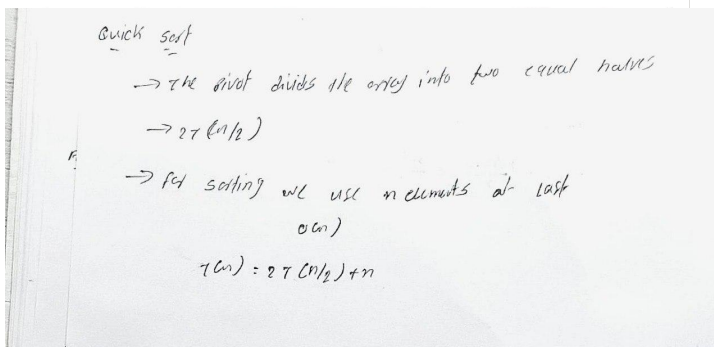
```
quickSort(arr, 0, n - 1);
```

```
cout << "Sorted array: "; for  
    (int i = 0; i < n; i++) { cout  
        << arr[i] << " ";  
    }  
    return 0;  
}
```

OUTPUT:

```
Enter size of array: 5  
Enter 5 elements: 7  
3  
8  
2  
0  
Sorted array: 0 2 3 7 8
```

TIME COMPLEXITY:



$$T(n) = 2T(n/2) + n \rightarrow (1)$$

$$n \rightarrow n/2 \text{ put in (1)}$$

$$T(n/2) = 2T(n/4) + n/2$$

$$T(n) = 2T(n/4) + n \rightarrow (2)$$

sub (2) in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$= 2^2 T(n/4) + \frac{2n}{2} + n$$

$$= 2^2 T(n/4) + 2n \rightarrow (3)$$

$$n \rightarrow n/4 \text{ put in (3)}$$

$$T(n/4) = 2^2 T(n/8) + 2(n/4) \rightarrow (4)$$

(4) in (3)

$$T(n) = 2^2 [2^2 T(n/8) + \frac{2n}{4}] + 2n$$

$$= 2^4 T(n/8) + \frac{2^3 n}{4} + 2n$$

$$= 2^4 T(n/8) + 4n$$

$$T(n) = 2^4 T(n/8) + 4n$$

$$T(n) = 2^4 T(n/8) + 4n$$

$$\frac{n}{2^4} = 1$$

$$n = 2^4$$

applying log both sides

$$\log n = \log 2^4$$

$$\log n = \log 2^4$$

$$T(n) = 2^{\log n} T(n/2^{\log n}) + \log n \log n$$

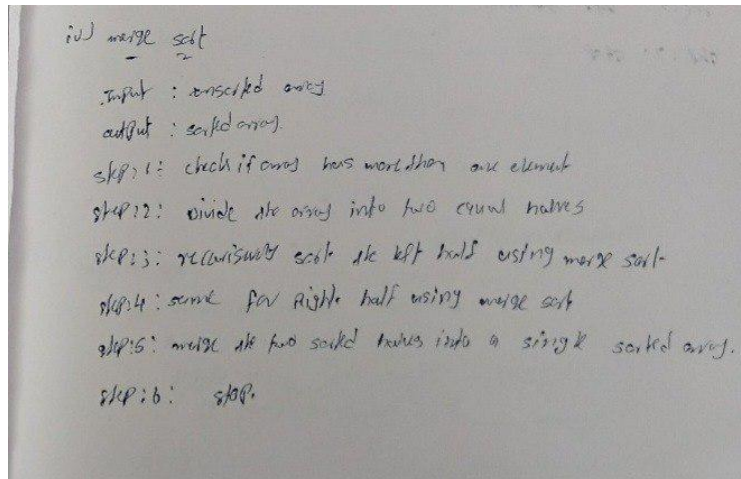
$$= n^{\log 2} \left(\frac{n}{n^{\log 2}} \right) + \log n \log n$$

$$= n \log n + n$$

$$T(n) = O(n \log n)$$

5.MERGE SORT

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

void merge(int arr[], int left, int mid, int right) { int
    n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++]; else
            arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() { int
    n;
    cout << "Enter number of elements: "; cin
    >> n;
    int arr[n];
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 5
Enter elements: 7
9
0
2
5
Sorted array: 0 2 5 7 9
```

TIME COMPLEXITY:

merge sort

→ the array is divided into two halves repeatedly
number of divisions: $\log_2 n$ ($2T(n/2)$)

→ merge: each level merges out n elements

Base Case

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2T(n/4) + n \rightarrow (1)$$

$n \rightarrow n/2$ put in (1)

$$T(n/2) = 2T(n/4) + n/2$$

$$T(n) = 2T(n/4) + n/2 \rightarrow (2)$$

sub (2) in (1)

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$= 2^2 T(n/4) + \frac{2n}{2} + n$$

$$= 2^2 T(n/4) + 2n \rightarrow (3)$$

$n \rightarrow n/4$ put in (3)

$$T(n/4) = 2^2 T(n/8) + 2(n/4) \rightarrow (4)$$

(4) in (3)

$$T(n) = 2^3 [2^2 T(n/8) + \frac{1n}{4}] + 2n$$

$$= 2^4 T(n/8) + \frac{2^3 n}{4} + 2n$$

$$= 2^4 T(n/8) + 4n$$

$$T(n) = 2^4 T(n/8) + 4n$$

$$T(n) = 2^4 T(n/8) + 4n$$

$$\frac{n}{2^4} \geq 1 \Rightarrow n \geq 2^4$$

apply log both sides

$$\log n = \log 2^4 \Rightarrow 4 \log 2$$

$$T(n) = 2^{\log_2 n} T(n/2^{\log_2 n}) + 10^3 n \log n$$

$$= n^{\log_2 2} - \left(\frac{n}{n^{\log_2 2}}\right) + 10^3 n \log n$$

$$= n \log n + n$$

$$T(n) = O(n \log n)$$

CONCLUSION:

Bubble Sort and Selection Sort are simple but inefficient with $O(n^2)$ in all major cases. Insertion Sort improves to $O(n)$ in the best case when the array is already sorted. Quick Sort is very efficient on average with $O(n \log n)$, though it can degrade to $O(n^2)$. Merge Sort is consistently $O(n \log n)$ in all cases but requires extra space.